Kathmandu University

Department of Computer Science and Engineering

Dhulikhel, Kavre

A Project Report

on

"Epidemic Spread Simulation"

[Code No : COMP 202]

(For partial fulfillment of II/I Year/Semester in Computer Science/Engineering)

Submitted by

Sushant Sharma Timilsina (Reg No. 038000-24)

Submitted to

Er. Sagar Aacharya

Department of Computer Science and Engineering

Submission Date:2/24/2026

Abstract

This project presents the design and implementation of an Epidemic Spread Simulator, a console based C++ application that models the propagation of an infectious disease through a social contact network. The primary objective was to apply core Data Structures and Algorithms (DSA) concepts, specifically Graphs, Breadth-First Search (BFS), and Queues in a practical, visually meaningful simulation. The population is represented as an undirected weighted graph using an adjacency list, where each node is a person and each edge is a contact between two individuals. Disease spread is modeled as a BFS traversal originating from a designated Patient Zero (The first person to be infected and thus spreading the disease). Health states such as Healthy, Infected, Recovered, and Quarantined are tracked per node. A quarantine mechanism allows selective node removal from the graph, demonstrating how severing connections affects propagation. The simulation outputs a day-by-day grid visualization and statistical summary to the terminal.

The project demonstrates that fundamental DSA constructs can directly model real-world systems. The BFS level-order traversal maps naturally to epidemic wave propagation, making the algorithm's behavior intuitively observable through simulation output.

Keywords: *Graph, Adjacency List, BFS, Queue, Epidemic Simulation, C++, Node State, Quarantine*

Acknowledgement

I would like to express my sincere gratitude to Er. Sagar Aacharya and the teaching staff of the Department of Computer Science and Engineering for their invaluable guidance, encouragement, and constructive feedback throughout the development of this mini project on Data Structures and Algorithms. I am also thankful to my institution for providing the resources and a supportive environment that enabled me to explore and apply theoretical concepts in practical implementation. My heartfelt appreciation goes to my peers and friends for their collaborative discussions and motivation, which enriched my learning experience. Finally, I am deeply grateful to my family for their constant support and encouragement, which kept me motivated to successfully complete this project.

Table of Contents

List of Figures

Acronyms/Abbreviations

DSA	Data Structures and Algorithms
BFS	Breadth First Search
FIFO	First In First Out
SIR	Susceptible Infected Recovered
GUI	Graphical User Interface
SFML	Simple and Fast Multimedia Library

Chapter 1 Introduction

This report documents the design and implementation of an Epidemic Spread Simulator, a mini project developed as part of the Data Structures and Algorithms course. The project applies fundamental DSA concepts, specifically Graphs, Breadth-First Search, and Queues to simulate how an infectious disease propagates through a population modeled as a social contact network. Implemented entirely in C++ using only the Standard Library, the simulator demonstrates how abstract algorithmic concepts directly map to observable, real-world phenomena.

1.1 Background

Epidemic modeling is an important application area in computational biology and public health. Understanding how diseases spread through populations requires modeling both the structural properties of social networks and the dynamic processes that occur on them. From a computer science perspective, this problem maps directly to graph traversal, one of the most fundamental topics in Data Structures and Algorithms.

This project applies those DSA concepts to build a functional epidemic simulation. Rather than using mathematical differential equations (as in traditional SIR models), the simulation uses a discrete graph-based approach where each individual is a node, each contact is an edge, and disease propagation follows BFS level-by-level, one day at a time

1.2 Objectives

- To model a social contact network as an undirected graph using an adjacency list.
- To implement epidemic spread using BFS traversal with a queue.
- To track per-node health states: Healthy, Infected, Recovered, Quarantined.
- To implement a quarantine mechanism by removing node edges from the graph.

1.3 Motivation and Significance

The motivation for this project was to move beyond abstract DSA exercises and apply graph and BFS concepts in a context that is visually engaging and conceptually meaningful. An epidemic simulator makes BFS behavior directly observable each wave of infection corresponds exactly to a BFS level which makes the algorithm's properties tangible and easy to explain

Chapter 2 DSA Concepts Used

2.1 Graph Adjacency List

The core data structure of this project is an undirected graph, implemented using an adjacency list. The population of n people is represented as n nodes, and contact between two individuals is represented as an undirected edge.

An adjacency list was chosen over an adjacency matrix because:

- Space efficiency: O(V + E) vs O(V^2) for a sparse social network
- Faster neighbor iteration: BFS only needs to visit neighbors, not all nodes
- More natural for dynamic graphs: removing edges (quarantine) is straightforward

In C++, the adjacency list is implemented as an array of vectors:

```
// LETS first create a graph:
class graph{
    private:
        int peoples;
        vector<int> adjlist[MAX_PEPS];

    public:
    // making a constructor
        graph (int n){
            peoples = n;
        }
        // a function to make the bidirectional edge
        void addEdge(int s, int d){
            // here s and d are two nodes between which a edge exists!!
            adjlist[s].push_back(d);
            adjlist[d].push_back(s);
        }
```

Fig 2.1: Graph implementation

2.2 Breadth-First Search (BFS)

BFS is the algorithmic heart of the simulation. The core insight is that epidemic spread and BFS traversal are structurally identical: an infected person spreads the disease to their direct contacts in one time step, exactly as BFS visits all neighbors of a node before moving deeper.

The key property used is BFS level-order traversal. Each level of BFS corresponds to one day of spread. This is achieved by recording the queue size at the start of each iteration, called level and processing exactly that many nodes before incrementing the day counter:

```
// adding the day counter since my model has become infect everyone script:
int day = 0;
int totalInfected = countState(peoples, Infected) + countState(peoples, Quarantine);

while(Queue.size()!=0){
    cout<<"Day :"<<day<<endl;

    // bug fixed; level shou;d be how many peps are there in queue not what day it is
    int level = Queue.size();

    for (int j =0 ; j< level ; j++){
    int source = Queue.front();
    Queue.pop();


    for (int neighbor : adjlist[source]){

    if(personstate[neighbor] == Healthy ){

        personstate[neighbor] = Infected;
        // added our destination node to the queue to make it a new source
        Queue.push(neighbor);
        totalInfected++;

        }

    }
    personstate[source] = Recovered;
    }
        printGrid();
        printDaySummary(day);
 day++;
}
printFinalStats(day, totalInfected);
```

Fig 2.2: BFS implementation

Without level, all nodes would be processed without any day boundaries. With it, the queue naturally separates into daily waves, mirroring real epidemic progression.

2.3 Queue

The STL queue is used to manage the BFS, the set of currently infected people whose neighbors have not yet been visited. The queue enforces the FIFO property that makes BFS work: nodes are processed in the order they were infected, ensuring that nodes closer to Patient Zero are processed before those further away.

The queue also serves a secondary purpose: it eliminates the need for a separate visited array. Since a node is only pushed into the queue when its state changes from Healthy to Infected, and the infection check always verifies the Healthy state, the same node can never be enqueued twice.

```
// BFS spread  implementation using queue
void bfs_spread(){

    queue<int> Queue;

    // finding the patient zero
    for(int i =0 ; i<peoples ; i++){

        if (personstate[i]== Infected){
            Queue.push(i);
        }
    }
```

Fig 2.3: Queue implementation

Chapter 3 Design and Implementation

3.1 System Overview

The simulator is a single-file C++ console application with no external library dependencies beyond the C++ Standard Library. It is structured into three logical layers: a utility layer of helper functions for display, a graph class containing all DSA logic, and a main function handling initialization and program flow.

3.2 Key Algorithms

- Building the Contact Network:

Algorithm Type: Random Graph Generation

This algorithm runs once at the start to build the social network. It decides who is connected to whom before any disease spreads.

Step 1. Go through every person in the population one by one.

Step 2. For each person, randomly decide how many contacts they will have (between 2 and 5).

Step 3. Repeat for the chosen number of contacts: pick a random person from the population.

Step 4. If that random person is not the same as the current person, create a connection between them. This connection goes both ways if A knows B, then B also knows A.

Step 5. Continue until all people have been assigned their contacts. The network is now ready for simulation.

- Disease Spread — BFS Level-by-Level

Algorithm Type: Breadth-First Search (BFS) on an Undirected Graph

Step 1. Mark Patient Zero as Infected and add them to the queue.

Step 2. While the queue is not empty, take a snapshot of how many people are currently in the queue. This snapshot is today's wave count.

Step 3. Process exactly that many people and no more. For each person taken out of the queue, look at all of their contacts.

Step 4. If a contact is still Healthy, mark them as Infected and add them to the back of the queue. They will be processed tomorrow.

Step 5. After processing all of their contacts, mark the current person as Recovered.

Step 6. Once all of today's people have been processed, print the grid and day summary, then move to the next day.

Step 7. Repeat from Step 2 until the queue is completely empty, meaning no more infected people remain.

- Quarantine

Algorithm Type: Graph Node Isolation (Edge Deletion)

Step 1. Select the person to be quarantined.

Step 2. Delete every entry in that person's contact list. They now have zero connections in the network.

Step 3. Change their state to Quarantined. From this point, BFS cannot spread the disease through them because they have no contacts, and their Quarantined state causes BFS to skip them if encountered.

Chapter 4 Results and Discussion

4.1 Sample Output

The following shows a representative run with n=30 people, seed= 42, Patient Zero set to Person 0 and Person 2 with Person 2 in quarantine .

```
                        DAY 0
H=Healthy  [I]=Infected  [R]=Recovered  [Q]=Quarantined


 0:[I]   1:[H]   2:[Q]   3: H    4: H 5: H   ...
10: H   11:[I]  12: H   ...


Day 0 Summary (Total: 30 people)
[H] Healthy    : 22 people
[I] Infected   :  6 people
[R] Recovered  :  1 people
[Q] Quarantine :  1 people


[ Simulation continues Day 1 through Day 3... ]


           SIMULATION COMPLETE
Total Days Taken  : 4
Total Ever Infected  : 30
Never Infected (safe): 0
Recovered         : 29
Quarantined       : 1
```

Fig 4.1: Sample Output

4.2 Analysis of Results

The simulation ran for 4 days with 30 people. Day 0 produced the first BFS wave from Patient Zero, infecting 6 of their direct neighbors immediately. Day 1 saw the largest single-day expansion as those 6 people spread to their own contacts. By Day 3, all reachable nodes had been infected and recovered, demonstrating BFS exhausting all paths in the graph.

Person 2, who was quarantined before the simulation began, had their adjacency list cleared. This person appears as [Q] throughout all days and is counted as Quarantined in the final stats rather than Recovered, directly showing the effect of graph structure modification on BFS reachability.

4.3 Complexity Analysis

Operation	Complexity	Explanation
BFS Spread	O(V + E)	Each node and edge visited at most once
addEdge()	O(1)	push_back on vector
countState()	O(V)	Linear scan of state array
Space (Adjacency List)	O(V + E)	Sparse graph; efficient vs $O(V^2)$ matrix

4.4 Challenges and Limitations

- No infection probability: The current model infects all healthy neighbors with 100% probability. A real epidemic model would use a probabilistic spread rate.
- No re-infection: Recovered nodes cannot be infected again. Adding this would require a separate visited array per simulation run.
- Terminal-only visualization: Output is console-based. A graphical library like SFML would significantly improve visual impact.

Chapter 5 Conclusion and Recommendation

5.1 Conclusion

This project successfully implemented an Epidemic Spread Simulator in C++ using three core DSA concepts: an undirected graph with an adjacency list, Breadth-First Search for disease propagation, and a queue as the BFS frontier. The simulation demonstrates that BFS level-order traversal directly models epidemic wave propagation; each day of spread corresponds exactly to one level of BFS.

The quarantine feature demonstrates a fundamental graph property: BFS can only traverse existing edges. Removing a node's edges (adjList[node].clear()) makes it completely unreachable and unable to spread infection, showing how graph structure directly determines algorithmic outcomes.

The state-based visited check using the person's health state instead of a separate boolean array is an elegant design decision that reduces memory usage while maintaining correctness. Overall, the project achieves its goal of applying DSA concepts to build a practical, meaningful, and visually demonstrable simulation.

5.2 Future Enhancements

- Infection probability: Add a parameter so each contact has a chance of not spreading the disease, making the model more realistic
- Graphical visualization using SFML: Render nodes as colored circles on screen, updating in real time as the simulation progresses
- Interactive quarantine in step mode: Allow the user to quarantine a person mid-simulation between days.

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press. [Graph representations, BFS algorithm, Chapter 22]
2. Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley. [Graph traversal and adjacency list implementation]
3. Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley. [STL containers: vector, queue; enum types]
4. Kermack, W. O., & McKendrick, A. G. (1927). A contribution to the mathematical theory of epidemics. Proceedings of the Royal Society of London, 115(772), 700-721. [SIR epidemic model foundations]